Zero to Hero Study Handbook: local-slm-ollama-benchmark

This handbook is built from static analysis of the repository files in `.` (no runtime execution in this write-up). It is designed to take a new learner from orientation to confident code-level understanding.

Module 1: Foundations & Architecture

High-level overview: what this project does

`local-slm-ollama-benchmark` is a Python CLI project for evaluating local Ollama models on one machine, then comparing quality and speed using deterministic rubrics.

Main use cases in this repo:

- Run a repeatable benchmark across exactly three local models via `local-slm-ollama-benchmark benchmark`.
- Produce machine-readable and human-readable artifacts for analysis:
  - `raw_results.json`
  - `prompt_runs.csv`
  - `model_summary.csv`
  - `benchmark_report.md`
- Run an interactive local chat session via `local-slm-ollama-benchmark chat`.

Core paradigms and patterns used here

Definitions first, then where each appears in code.

- CLI command pattern: a command-line parser routes subcommands to handler functions.
  - Used in `src/local_slm_ollama_benchmark/cli.py` via `argparse`, `set_defaults(handler=...)`, and `main()`.
- Async I/O: network calls are non-blocking with `async/await`.
  - Used in `OllamaClient` (`httpx.AsyncClient`) and async benchmark/chat paths.
- Schema-first configuration: config is validated into typed models before use.
  - Used in `config.py` with Pydantic models (`BenchmarkConfig`, `PromptCase`, etc.).
- Data-pipeline style batching: benchmark flow loops through models, prompt cases, and repeated iterations, then aggregates.
  - Implemented in `BenchmarkRunner.run()` in `benchmark.py`.
- Deterministic heuristic scoring: response quality is measured with explicit rubrics, not an external judge model.

  - Implemented in `quality.py` (`keywords`, `json_keys`, `exact_match` + optional `max_words` penalty).
- Artifact-first observability: every run emits JSON/CSV/Markdown outputs for auditing.
  - Implemented by `_write_prompt_csv`, `_write_model_csv`, `_write_markdown_report` in `benchmark.py`.

Architecture description: key components and interaction

Core components:

- CLI and entrypoint:
  - `pyproject.toml` script: `local-slm-ollama-benchmark = "local_slm_ollama_benchmark.cli:main"`
  - `src/local_slm_ollama_benchmark/__main__.py` imports and calls `main()`
- Config layer:
  - `src/local_slm_ollama_benchmark/config.py` validates TOML config into typed models
- Ollama adapter:
  - `src/local_slm_ollama_benchmark/ollama_client.py` wraps `/api/tags` and `/api/generate`
- Benchmark engine:
  - `src/local_slm_ollama_benchmark/benchmark.py` orchestrates runs, scoring, summaries, reports
- Quality + metrics helpers:
  - `src/local_slm_ollama_benchmark/quality.py`
  - `src/local_slm_ollama_benchmark/metrics.py`
- Input and outputs:
  - Input config: `configs/benchmark.toml`
  - Output artifacts: `artifacts/<run_id>/...`

Main benchmark flow (ASCII):

```
CLI (benchmark subcommand)
    |
    v
load_benchmark_config(configs/benchmark.toml)
    |
    v
BenchmarkRunner.run(output_root, run_name)
    |
    +--> OllamaClient.list_models()  --> validates configured model names
    |
    +--> for each model:
    |       warmup -> OllamaClient.generate()
    |       for each prompt case:
```

```
    |           for each iteration:
    |               OllamaClient.generate()
    |               evaluate_response()
    |               build PromptRunRow
    |
    +--> summarize (_summarize_models)
    |       - latency, p95, tokens/sec, quality
    |       - balanced_score = 0.6*quality_norm + 0.4*speed_norm
    |
    +--> write artifacts
            raw_results.json
            prompt_runs.csv
            model_summary.csv
            benchmark_report.md
```

Main chat flow (ASCII):

```
CLI (chat subcommand)
    |
    v
_run_chat(args) -> GenerationConfig
    |
    v
OllamaClient.list_models() validation
    |
    v
REPL loop:
  read user input
  call OllamaClient.generate()
  print response + latency/tokens-per-second
```

Module 2: Repository Map

Focus: files a new contributor should understand first.

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`README.md`	Project overview, quickstart, sample commands, tradeoff context	N/A	CLI examples for `benchmark` and `chat`
`pyproject.toml`	Package metadata, dependencies, CLI entrypoint	N/A	`[project.scripts]` `local-slm-ollama-benchmark`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`.python-version`	Preferred local interpreter version for dev tooling	N/A	`3.14`
`configs/benchmark.toml`	Benchmark input definition	N/A	`ollama_host`, `models`, `[generation]`, `[runtime]`, `[cost]`, `[[prompts]]`
`src/local_slm_ollama_benchmark/__main__.py`	Package execution entrypoint	main import and invocation	N/A
`src/local_slm_ollama_benchmark/cli.py`	CLI parser and command handlers	main, _build_parser, _run_benchmark, _run_chat	`--config`, `--output-dir`, `--run-name`, `--repeat-count`, chat args
`src/local_slm_ollama_benchmark/config.py`	Dataclasses + TOML loader/validation	PromptCase, GenerationConfig, RuntimeConfig, CostConfig, BenchmarkConfig, load_benchmark_config	Validation rules for prompt fields and model count
`src/local_slm_ollama_benchmark/ollama_client.py`	Async HTTP wrapper over Ollama APIs	GenerateResponse, OllamaClient.list_models, OllamaClient.generate	Request payload fields: model, prompt, options, keep_alive, think
`src/local_slm_ollama_benchmark/benchmark.py`	End-to-end benchmark orchestration and artifact writers	BenchmarkRunner.run, _run_single_prompt, _summarize_models, collect_system_info	balanced_score, cost estimation formula, artifact filenames
`src/local_slm_ollama_benchmark/quality.py`	Deterministic quality rubric scoring	QualityResult, evaluate_response	evaluation_type, expected_keywords, required_json_keys, expected_answer, max_words
`src/local_slm_ollama_benchmark/metrics.py`	Unit conversion helpers	ns_to_sec, tokens_per_second, percentile	Percentile interpolation logic

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`tests/test_quality.py`	Unit tests for rubric behavior	`test_keyword_quality_scoring_hits_all_keywords`, etc.	Validates keyword/json/exact-match behavior
`tests/test_metrics.py`	Unit tests for metrics utilities	`test_ns_to_sec`, `test_tokens_per_second`, `test_percentile_linear_interpolation`	Checks conversion, throughput computation, and linear interpolation
`artifacts/*/`	Saved benchmark outputs from previous runs	N/A	`raw_results.json`, `prompt_runs.csv`, `model_summary.csv`, `benchmark_report.md`
`notebooks/local_slm_ollama_tutorial.ipynb`	Learning notebook that maps to production code	`run_cmd` helper (not production runtime)	`RUN_BENCHMARK` flag, paths for config/artifacts

Module 3: Core Execution Flows

Flow 1: Benchmark run (`benchmark` subcommand)

Step-by-step path through real code

1. CLI entrypoint:
   - `pyproject.toml` maps command to `local_slm_ollama_benchmark.cli:main`.
   - `main()` builds parser, selects subcommand handler, configures logging.
2. Benchmark handler:
   - `_handle_benchmark()` calls `asyncio.run(_run_benchmark(args))`.
3. Config loading:
   - `_run_benchmark()` calls `load_benchmark_config(args.config)`.
   - If `--repeat-count` is provided, it overrides `config.runtime.repeat_count` using `model_copy(update=...)`.
4. Runner orchestration:
   - `BenchmarkRunner(config).run(output_root, run_name)`.
   - Run directory is `output_root / run_id`, where `run_id` is explicit `--run-name` or UTC timestamp.
5. Model availability check:
   - `OllamaClient.list_models()` calls `GET /api/tags`.
   - Missing configured models raise a `RuntimeError`.
6. Warmup + measurement loop:
   - For each model: `_warmup_model(...)` calls `OllamaClient.generate(...)` with warmup prompt.
   - For each prompt case and iteration: `_run_single_prompt(...)` calls generate, measures wall time, scores output, creates `PromptRunRow`.

7. Aggregation:
    - `_summarize_models(...)` groups rows per model and computes:
        - `avg_latency_sec`, `p95_latency_sec`
        - `avg_tokens_per_sec`
        - `avg_quality_score`
        - `quality_delta_vs_baseline`
        - `speedup_vs_baseline`
        - `balanced_score = 0.6 * quality_norm + 0.4 * speed_norm`
        - Optional `estimated_benchmark_cost_usd`
8. Artifact writing:
    - `raw_results.json` via `json.dumps(payload, indent=2)`
    - `prompt_runs.csv` via `_write_prompt_csv`
    - `model_summary.csv` via `_write_model_csv`
    - `benchmark_report.md` via `_write_markdown_report`

Short code fragment (from `cli.py`) showing handoff:

```
def _handle_benchmark(args: argparse.Namespace) -> None:
    asyncio.run(_run_benchmark(args))

async def _run_benchmark(args: argparse.Namespace) -> None:
    config = load_benchmark_config(args.config)
    runner = BenchmarkRunner(config)
    artifacts = await runner.run(output_root=args.output_dir, run_name=args.run_name)
```

Exact input shape for benchmark config The validated top-level shape in `BenchmarkConfig` is:

- `ollama_host: str`
- `models: list[str]` (must be unique, and exactly length 3)
- `generation: GenerationConfig`
    - `temperature: float`
    - `top_p: float`
    - `num_predict: int`
    - `think: bool | str | None`
    - `seed: int`
- `runtime: RuntimeConfig`
    - `repeat_count: int`
    - `request_timeout_sec: float`
    - `keep_alive: str`
    - `warmup_prompt: str`
- `cost: CostConfig`
    - `electricity_rate_usd_per_kwh: float | None`
    - `assumed_power_watts: float | None`
- `prompts: list[PromptCase]` where each prompt has:
    - `id`, `title`, `prompt`, `evaluation_type`
    - `expected_keywords` (for `keywords`)

- `required_json_keys` (for `json_keys`)
- `expected_answer` (for `exact_match`)
- optional `max_words`

Exact output shapes produced by benchmark flow `raw_results.json` top-level keys (from `benchmark.py` and artifact examples):

- `run_id: str`
- `started_at_utc: str` (ISO 8601)
- `completed_at_utc: str` (ISO 8601)
- `ollama_host: str`
- `config: dict` (serialized validated config)
- `system: dict[str, str]`
- `prompt_runs: list[dict]`
- `model_summary: list[dict]`
- `tradeoff_notes: list[str]`

One `prompt_runs` item shape:

- `model`, `case_id`, `case_title`, `iteration`
- `wall_time_sec`
- `total_duration_sec`, `load_duration_sec`, `prompt_eval_duration_sec`, `eval_duration_sec`
- `eval_tokens`, `tokens_per_sec`
- `quality_score`, `output_words`
- `response`
- `quality_details` (rubric diagnostics)

`prompt_runs.csv` columns:

- `model, case_id, case_title, iteration, wall_time_sec, total_duration_sec, load_duration_sec, prompt_eval_duration_sec, eval_duration_sec, eval_tokens, tokens_per_sec, quality_score, output_words`

`model_summary.csv` columns:

- `model, sample_count, avg_latency_sec, p95_latency_sec, avg_tokens_per_sec, avg_quality_score, quality_delta_vs_baseline, speedup_vs_baseline, balanced_score, estimated_benchmark_cost_usd`

Flow 2: Chat run (`chat` subcommand)

Step-by-step path through real code

1. `main()` dispatches to `_handle_chat()`.
2. `_handle_chat()` calls `asyncio.run(_run_chat(args))`.
3. `_run_chat()` builds `GenerationConfig` from CLI flags.
4. `think` argument is normalized:
   - `"false"` becomes `False`
   - `"true"`, `"low"`, `"medium"`, `"high"` stay string values.

5. `OllamaClient.list_models()` validates `--model` exists locally.
6. REPL loop:
    - read user input
    - call `OllamaClient.generate(...)`
    - print response text and metrics using `ns_to_sec` and `tokens_per_second`.

Short code fragment (from `cli.py`) showing model validation:

```
installed_models = await client.list_models()
if args.model not in installed_models:
    raise ValueError(
        f"Model '{args.model}' is not available. Installed models: {installed_models}"
    )
```

Flow 3: Quality evaluation internals

`evaluate_response(case, response_text)` in `quality.py`:

- For `keywords`:
    - counts case-insensitive keyword hits and computes ratio.
- For `json_keys`:
    - parses full text as JSON, or extracts first balanced JSON object and parses it.
    - checks required keys present in parsed dict.
- For `exact_match`:
    - strict equality vs `expected_answer` after `.strip()`.
- If `max_words` is set:
    - computes word count.
    - applies verbosity penalty when exceeded:
        * `brevity_penalty = max_words / word_count`
        * `final_score *= brevity_penalty`

Return type is `QualityResult`:

- `score: float` (clamped to [0, 1], rounded to 4 decimals)
- `details: dict[str, float | int | bool]`

Flow 4: Metrics and summarization details

Metric helpers in `metrics.py`:

- `ns_to_sec(duration_ns)` converts nanoseconds to seconds.
- `tokens_per_second(eval_count, eval_duration_ns)` returns `None` for invalid/missing values.
- `percentile(values, p)` uses linear interpolation (validated by `tests/test_metrics.py`).

Summarization in `benchmark.py`:

- Baseline model is `config.models[0]`.

- `speedup_vs_baseline = baseline_avg_latency / model_avg_latency`.
- Summary rows are sorted by `balanced_score` descending.

Module 4: Setup & Run Guide

1) Prerequisites

- OS: Linux/macOS with Ollama running locally.
- Python:
  - `pyproject.toml`: `requires-python = ">=3.11"`
  - `.python-version`: `3.14`
- Package manager: `uv`.
- Local models: must match names in `configs/benchmark.toml`.
  - Current config lists:
    - `functiongemma:270m`
    - `phi3.5:3.8b`
    - `granite4.1:3b`

2) Install dependencies

```
uv sync
```

Dependencies from `pyproject.toml`:

- Runtime: `httpx`, `pydantic`, `rich`
- Dev: `pytest`

3) Environment variables and config files

- Required `.env` keys in this repository: none found in source/config files.
- Primary runtime config file: `configs/benchmark.toml`.
- Optional cost settings are in `[cost]` section of TOML:
  - `electricity_rate_usd_per_kwh`
  - `assumed_power_watts`

4) Typical command sequences

Install model weights in Ollama (examples from README):

```
ollama pull functiongemma:270m
ollama pull phi3.5:3.8b
ollama pull granite4.1:3b
```

Run benchmark:

```
uv run local-slm-ollama-benchmark benchmark \
  --config configs/benchmark.toml \
  --output-dir artifacts \
  --run-name my_run_name
```

Run chat:

```
uv run local-slm-ollama-benchmark chat \
  --model phi3.5:3.8b \
  --think false
```

Optional Python module entrypoint:

```
uv run python -m local_slm_ollama_benchmark
```

5) Migrations, seeding, and external services

- Database migrations: none (no database layer in repo).
- Seeding scripts: none.
- External service required at runtime: local Ollama server (`ollama_host`, default `http://127.0.0.1:11434`).

6) Export this handbook to PDF

From repository root:

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

Ordered study plan

1. Read `README.md` to understand project intent, commands, and artifact expectations.
2. Read `configs/benchmark.toml` to see concrete benchmark inputs and rubrics.
3. Read `src/local_slm_ollama_benchmark/config.py` to understand what config is legal and why.
4. Read `src/local_slm_ollama_benchmark/cli.py` for entrypoint routing and command behavior.
5. Read `src/local_slm_ollama_benchmark/ollama_client.py` for API payload/response boundaries.
6. Read `src/local_slm_ollama_benchmark/quality.py` and `metrics.py` for scoring and numeric helpers.
7. Read `src/local_slm_ollama_benchmark/benchmark.py` end-to-end for orchestration and artifact writing.
8. Read `tests/test_quality.py` and `tests/test_metrics.py` to confirm intended behavior.
9. Use `artifacts/real_run_20260612_v2/*` as a concrete reference for output schemas.
10. Optionally read `notebooks/local_slm_ollama_tutorial.ipynb` for guided learning mapped to production files.

Practice exercises (with file anchors)

1. Trace request path: starting at CLI, list every function call until the first `POST /api/generate` call in benchmark mode.
2. Config integrity: explain why this project enforces exactly three models and unique model names.
3. Rubric behavior: for a `keywords` case with 4 required terms and 3 hits, what is base score?
4. Verbosity penalty: if `max_words=80` and response has 120 words, what multiplier is applied?
5. JSON extraction: explain how `json_keys` mode can still succeed when JSON is wrapped inside plain text.
6. Baseline math: where does `speedup_vs_baseline` come from, and which model is baseline?
7. Cost output conditions: when is `estimated_benchmark_cost_usd` `null` vs numeric?
8. Chat safety check: what exact condition raises a `ValueError` before chat loop starts?
9. Artifact contract: list all required files that one benchmark run writes and which function writes each.
10. Quality/speed ranking: identify where summary rows are sorted and by what key.

Solution outlines

1. `cli.main()` -> `_handle_benchmark()` -> `_run_benchmark()` -> `BenchmarkRunner.run()` -> `_run_single_prompt()` -> `OllamaClient.generate()` (POST `/api/generate`).
2. `BenchmarkConfig._validate_models()` enforces uniqueness and exact length `3`; violations raise `ValueError`.
3. Base score is `3/4 = 0.75` in `evaluate_response()` keyword branch.
4. Multiplier is `80/120 = 0.6667` (approximately), applied as `final_score *= brevity_penalty`.
5. `_try_parse_json()` first tries `json.loads(text)`; if that fails it uses `_extract_first_json_object()` and retries parsing.
6. In `_summarize_models()`, baseline is `config.models[0]`; `speedup_vs_baseline = baseline_latency / avg_latency`.
7. Numeric only when both `cost.electricity_rate_usd_per_kwh` and `cost.assumed_power_watts` are set; otherwise `None`/empty CSV cell.
8. In `_run_chat()`, if `args.model` not in `await client.list_models()`, it raises `ValueError`.
9. `raw_results.json` (direct write in `run()`), `prompt_runs.csv` (`_write_prompt_csv`), `model_summary.csv` (`_write_model_csv`), `benchmark_report.md` (`_write_markdown_report`).
10. `_summarize_models()` ends with `summary_rows.sort(key=lambda row: row["balanced_score"], reverse=True)`.

Learner Verification Checklist

Use this checklist after you finish the study plan.

- Can you explain the end-to-end benchmark path from CLI parsing to artifact generation?
- Can you describe the exact schema constraints in `BenchmarkConfig` and `PromptCase`?
- Can you explain how each `evaluation_type` in `quality.py` computes score?
- Can you compute `tokens_per_second`, percentile behavior, and `balanced_score` logic manually?
- Can you describe every column in `prompt_runs.csv` and `model_summary.csv`?
- Can you explain why model availability is validated before benchmark/chat execution?
- Can you identify where hardware/system metadata is collected and persisted?
- Can you explain what changes are needed to add a new prompt case safely?
- Can you run through the chat loop logic mentally, including exit conditions and model checks?
- Can you explain what this project does not include (no DB, no migrations, no env-secret dependency)?